פרויקט גמר במגמת אלקטרוניקה ומחשבים

בית ספר אורט מלטון בת ים

שם פרויקט: חממה מבוקרת מרחוק

מגיש: טיגראן דוד סבין

תעודת זהות:331612598

בהנחיית: המורה יעקוב ברסקי

שנה:2026

תוכן עניינים

מבוא, דף תודות----------(3)
תרשים זרימה----------(4-5)
תרשים מלבנים----------(6)
רשימת רכיבים----------(7)
תקשורות----------(7-10)
סכמה חשמלית----------(11)
ESP32----------(12-14)
DHT22----------(15-17)
TFT TOUCHSCREEN----------(18-21)
WATER LEVEL SENSOR----------(22-24)
LDR----------(25-29)
RAINDROPS SENSOR----------(30-32)
FAN MOTOR----------(33-35)
קוד הפרויקט----------(36-46)
מדידות----------(47-50)
תיעוד----------(51-54)
סיכום ומסכנות, קשיים----------(55)
ביבליוגרפיה----------(56)

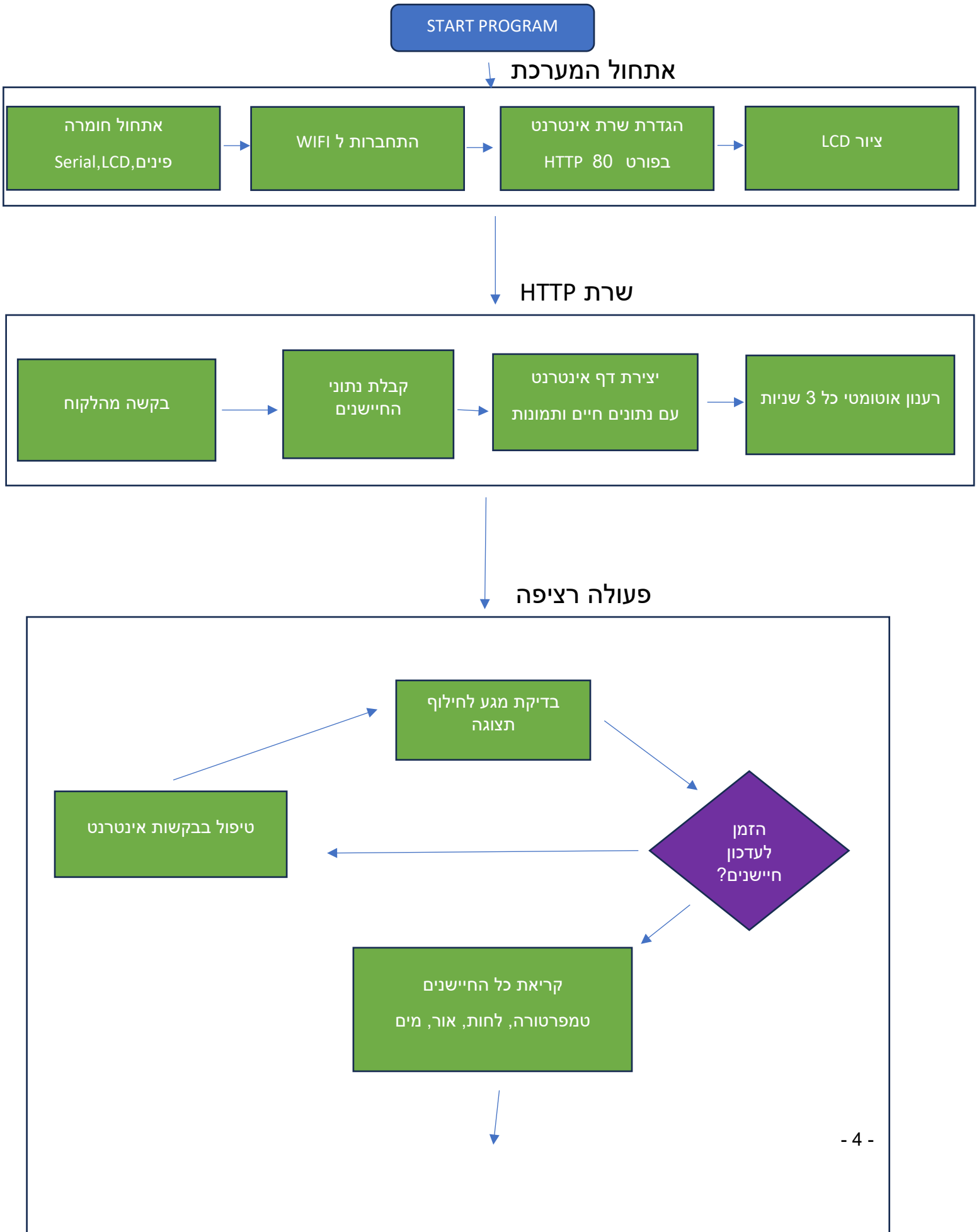
דף תודות

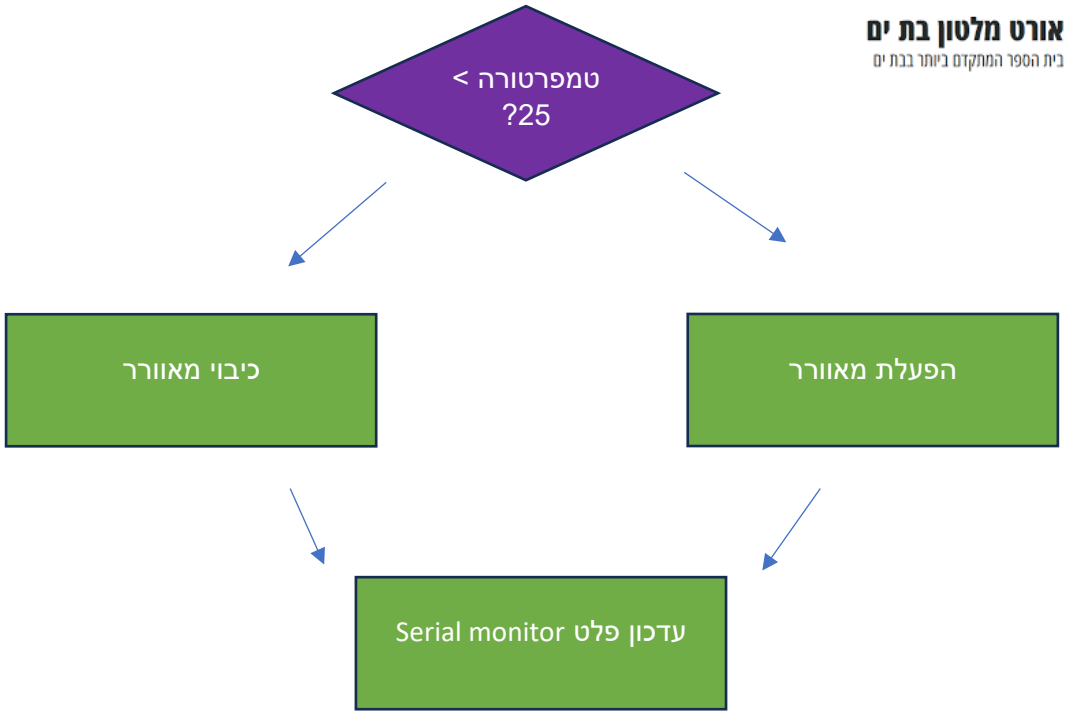
אני רוצה להודות למנהלת הבית ספר, עליזה אפרים על התמיכה והעידוד לאורך הדרך.
תודה לרכז מגמת אלקטרוניקה, רפי אוזנה על מה שלימד אותנו. תודה למחנך הכיתה, גיא שחינך אותנו להיות תלמידים טובים יותר.
תודה למנחה, יעקב ברסקי על הליווי והתמיכה לאורך הפרויקט.

מבוא לפרויקט

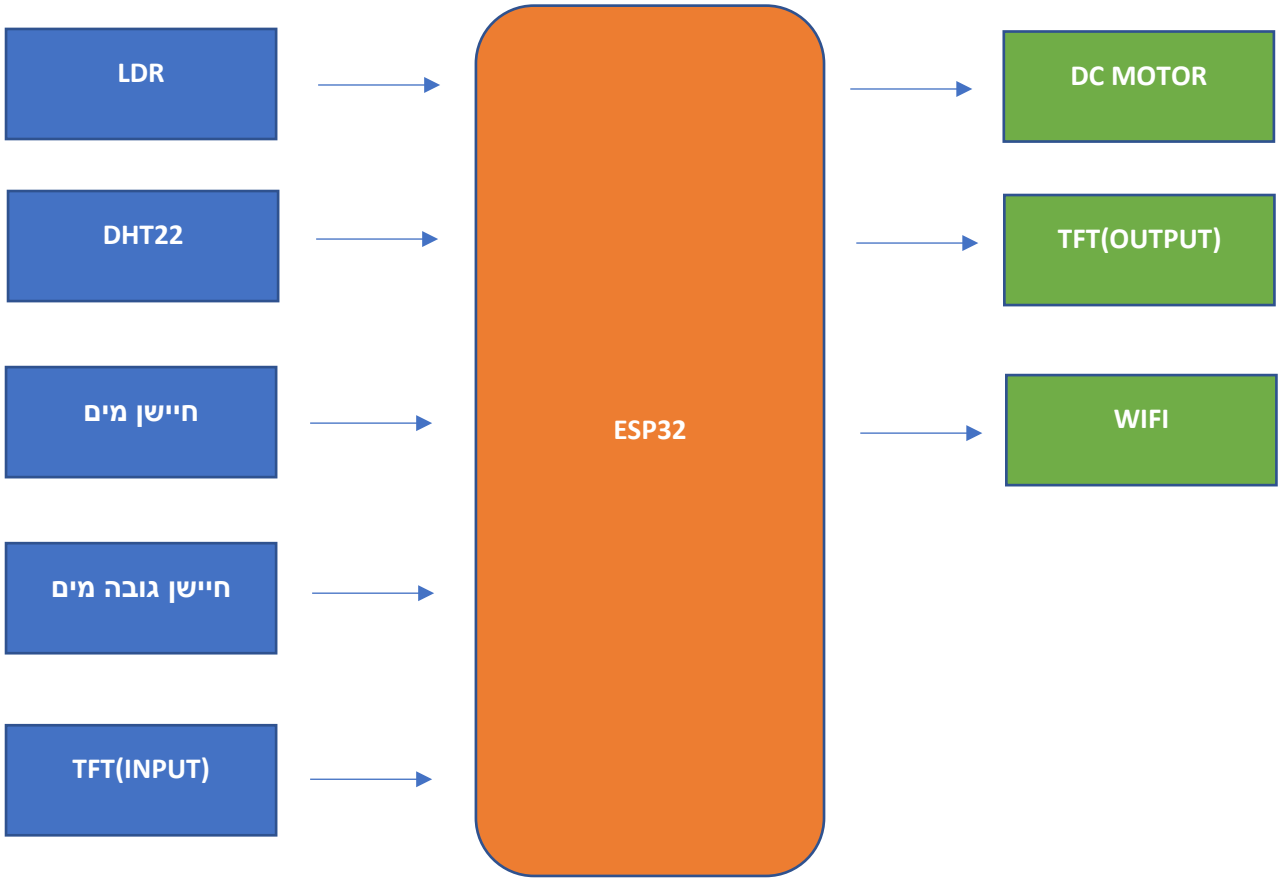
הפרויקט מתמקד בפיתוח מערכת אינטגרטיבית לניהול חממה חכמה, הממנפת טכנולוגיות IoT (Internet of Things) ליצירת פלטפורמת ניטור ובקרה אוטונומית. המערכת מנטרת פרמטרים קריטיים כגון טמפרטורה, לחות, רמת אור ומצב השקיה באמצעות מערך חיישנים מתקדם, ומבצעת התערבות אוטומטית מבוססת-אלגוריתמים (כגון הפעלת מערכות קירור או השקיה) בהתאם לנתונים בזמן אמת. בנוסף, המערכת מספקת ממשק ניהול מרחוק דרך שרת אינטרנט, המאפשר לחקלאים גישה לתצוגה ויזואלית של הנתונים ולשליטה ידנית מכל מקום. פתרון זה מטפל באתגרי הנצילות והבזבוז המשאבי שבשיטות המסורתיות, ומציע אלטרנטיבה סקיילבילית, חסכונית ומותאמת אישית לצרכים של חקלאים מודרניים — מקטנים עד בינוניים — במגזר החקלאי המתפתח.

תרשים זרימה


START PROGRAM
אתחול המערכת
אתחול חומרה
Serial,LCD,פינים
התחברות ל WIFI
הגדרת שרת אינטרנט
בפורט 80 HTTP
ציור LCD
שרת HTTP
בקשה מהלקוח
קבלת נתוני
החיישנים
יצירת דף אינטרנט
עם נתונים חיים ותמונות
רענון אוטומטי כל 3 שניות
פעולה רציפה
בדיקת מגע לחילוף
תצוגה
הזמן
לעדכון
חיישנים?
טיפול בבקשות אינטרנט
קריאת כל החיישנים
טמפרטורה, לחות, אור, מים


טמפרטורה > 25?
כיבוי מאוורר
הפעלת מאוורר
עדכון פלט Serial monitor

תרשים מלבנים



רשימת רכיבים:

ESP32

Touch Screen(TFT)

LDR Module

DHT22

חיישן מים

חיישן גובה מים

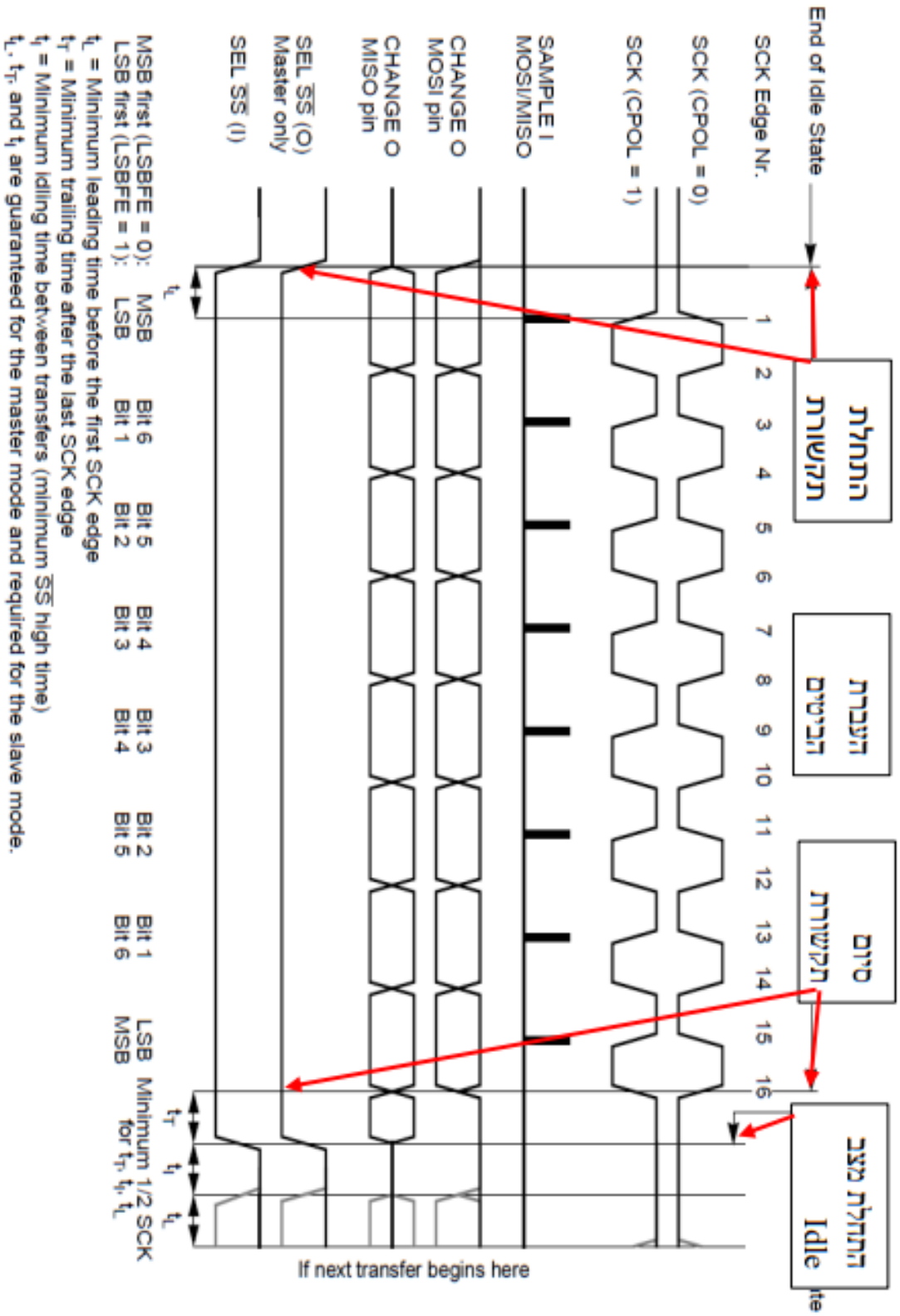
DC MOTOR

תקשורות:

SPI

תקשורת SPI היא פרוטוקול שפותח על ידי חברת מוטורולה בשנות ה80 , הפרוטוקול מתאפיין בכך שמשתמש ב4 קווי תקשורת לעומת .C2I העברת הנתונים מתבצעת באופן מתוזמן באמצעות רגל השעון ,(SCK) המאסטר שולח את הנתונים על רגל (MOSI) וקורא נתונים שמקבל מהרכיב שמחובר אליו ברגל (MISO) ובאמצעות הרגל הרביעית (SS) הוא בוחר איזה מן הכתובות הוא שולח/מקבל מקבלים ונים מאפיינים:

- תקשורת סידרתית - העברת הנתונים מתוזמנת באמצעות השעון.
- תקשורות מהירה - מהירות של עד 10 מגה לשנייה.
- 4 קווי תקשורת.
- קו נפרד שבוחר את הסלייב.
- ניתן לחבר כמות גדולה של רכיבים בגלל רגל ה SS שבוחרת איזה סלייב להשתמש לעומת C2I שצריכה להשתמש בכתובות זיהוי .
- הנתונים של MISO ו MOSI מועברים בו זמנית.



t_L = Minimum leading time before the first SCK edge

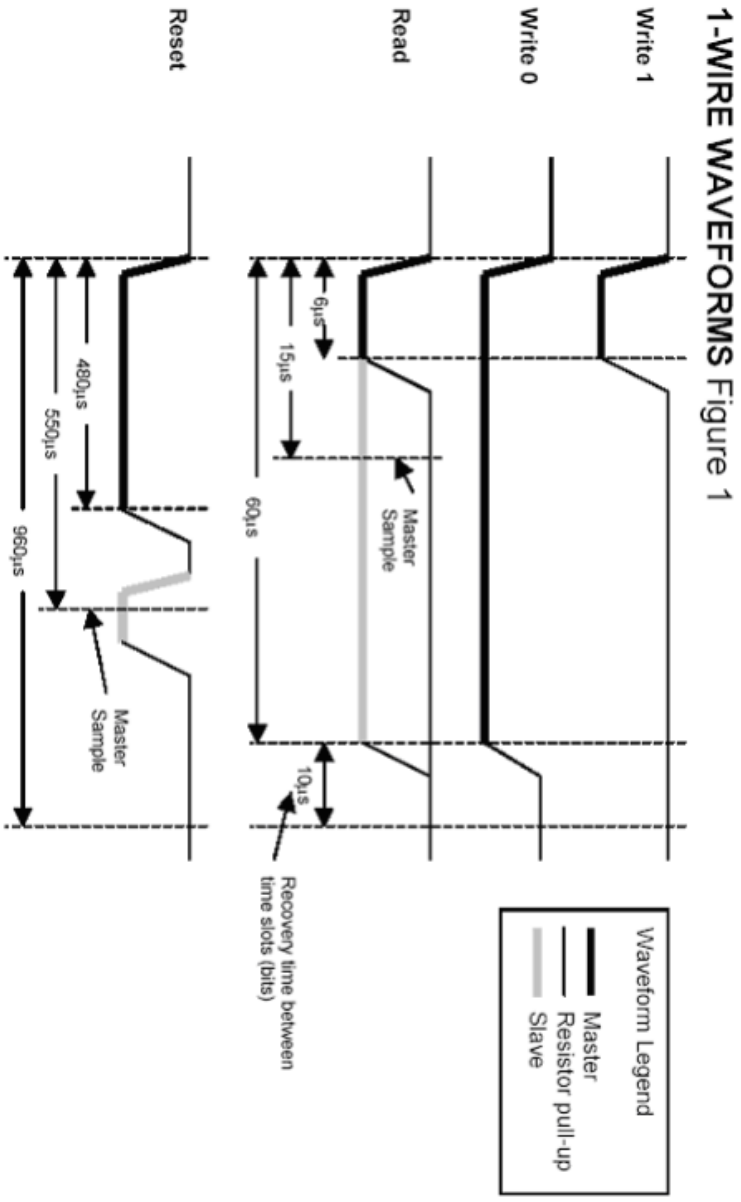
t_T = Minimum trailing time after the last SCK edge

t_I = Minimum idling time between transfers (minimum $\overline{SS}$ high time)

t_L, t_T, and t_I are guaranteed for the master mode and required for the slave mode.

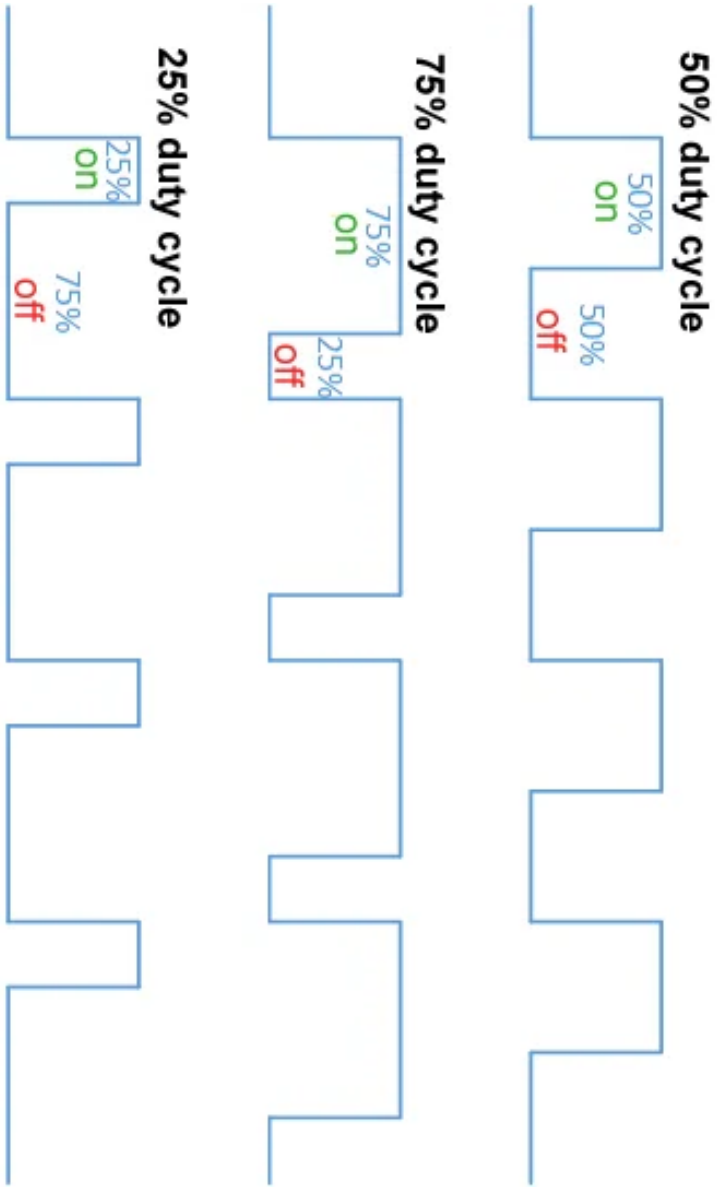
1 WIRE

תקשורת ONE WIRE היא פרוטוקול שפותח על ידי חברת Dallas Semiconductor המשתמש בחוט תקשורת יחיד בלבד להעברת נתונים דו-כיוונית, חסכוני בפינים אך איטי יחסית (עד Kbps16.3). הפרוטוקול מתאפיין בזיהוי ייחודי של כל רכיב באמצעות כתובת ROM בת 64 ביט, ומאפשר חיבור מספר גדול של רכיבים על קו אחד עם חיווט מינימלי. הוא משמש בעיקר לחיישנים כמו DHT22 ו-DS18B20, ודורש תזמון פולסים מדויק אך מספק פתרון פשוט ויעיל ליישומים שאינם דורשים מהירות גבוהה.

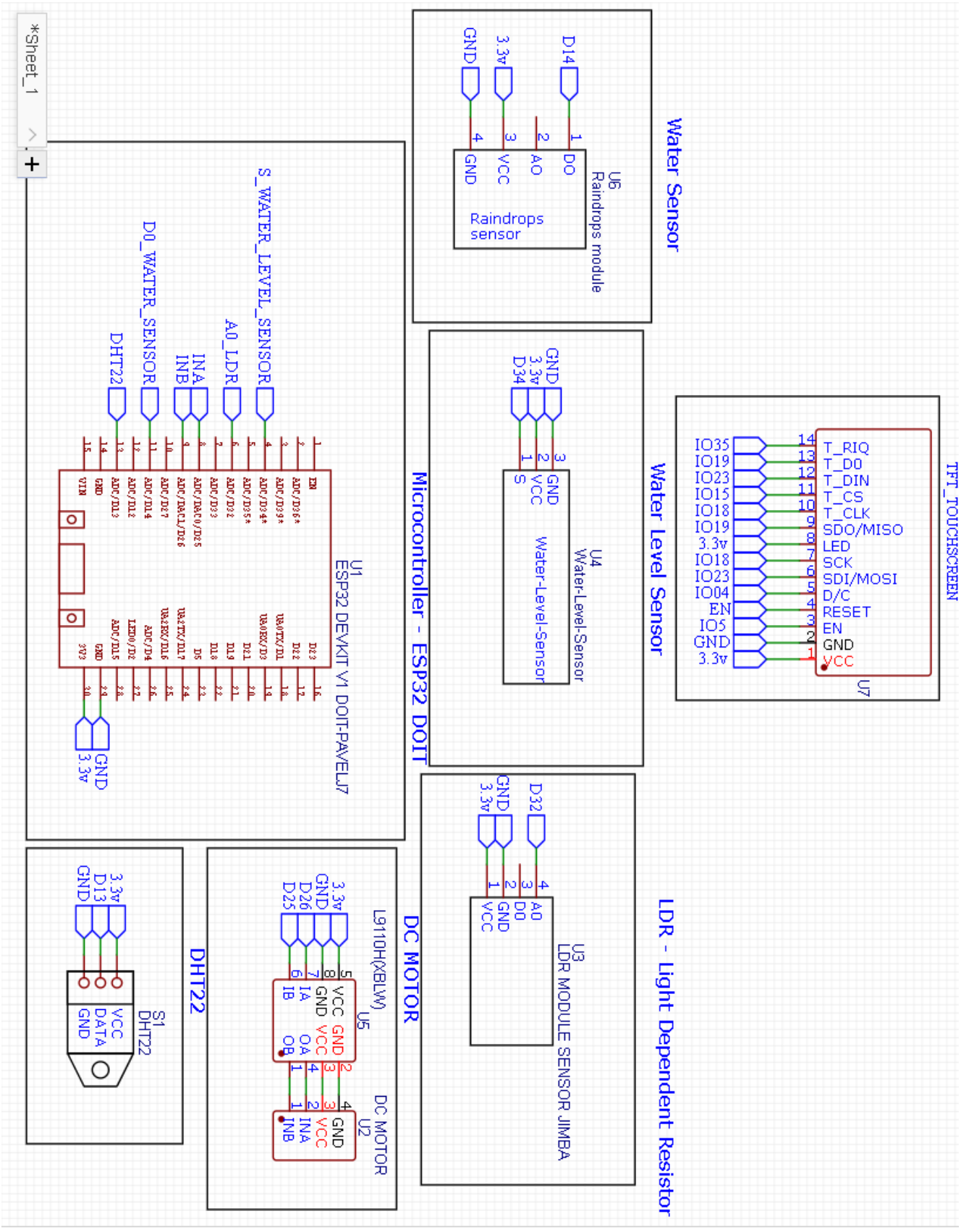


PWM

PWM (ראשי תיבות של Pulse Width Modulation, או בעברית: אפנון רוחב פולס) היא שיטה לשליטה בהספק חשמלי שמועבר לרכיב, באמצעות שינוי היחס בין זמן ה"הדלקה" לזמן ה"כיבוי" של אות מחזורי. במקום לשנות את המתח עצמו, שומרים על מתח קבוע אך משנים את משך הזמן שבו האות פעיל בכל מחזור (Duty Cycle).
לדוגמה, אם האות פעיל חצי מהזמן וכבוי חצי מהזמן, מתקבל הספק ממוצע של כ־50%. שיטה זו יעילה מאוד מבחינה אנרגטית, ולכן נפוצה בבקרת מהירות מנועים, עמעום נורות LED, ספקי כוח ממותגים ועוד.



סכמה חשמלית כוללת



תיאור הבקר בפרויקט

ESP32

"ESP32-WROOM-32D"הוא מודול קומפקטי, רב עוצמה וחסכוני באנרגיה שפותח על ידי
" Espressif Systems"

המודול מבוסס על השבב "ESP32-D0WD" מעבד כפול-ליבה 32-ביט, המיוצר על ידי "TSMC" טאיוואן.

הוא נמצא בשימוש נרחב ביישומי ,"IoT" בתים חכמים, אלקטרוניקה לבישה ויישומים תעשייתיים, בזכות השילוב בין ביצועים גבוהים, קישוריות אלחוטית וצריכת אנרגיה נמוכה.

מאפיינים עיקריים

מעבד(MCU):

מיקרו-מעבד 32-ביט ,"Xtensa LX6" כפול-ליבה.

מהירות שעון: 160–240 MHz.

מעבד "ULP Ultra Low Power" לחיסכון באנרגיה.

- **קישוריות אלחוטית:**
  "WIFI" בתדר: , 2.4 GHz מהירות עד 150_Mbps
  (תקן" b/g/n 802.11")
  "Bluetooth" גרסה 4.2 (RB/EDR + BLE)
- **זיכרון:**

  זיכרון פנימי "SRAM" 520 KB

  "ROM" 448 KB

  תמיכה בזיכרון פלאש חיצוני – המודול מגיע עם "SPI Flash" 4MB
- **ניהול צריכת חשמל:**

  מספר מצבי עבודה: פעיל, שינה עם מודם, שינה עמוקה.

  מותאם במיוחד ליישומי "IoT" חסכוניים באנרגיה.
- **קווי קלט/פלט:(GPIO)**

  עד 34 פינים לשימוש כללי.

  תמיכה בפרוטוקולים: "1-Wire","PWM", "I²S", "I²C", "SPI", "UART".

- **<u>ממירים וחיישנים:</u>**
  עד 18 ערוצי "ADC" (12 ביט)
  2 ערוצי "DAC" (8 ביט)

  10 כניסות חיישני מגע קיבוליים.

- **<u>אבטחה:</u>**

האצת חומרה לאלגוריתמים קריפטוגרפיים("AES", "SHA", "RSA", "ECC")

תמיכה ב- "Secure Boot" והצפנת פלאש.

- **<u>מתח עבודה:</u>**

3.0–3.6V (טיפוסי: 3.3V)

- **<u>פיזי</u>**:

ממדים: 3.1 × 25.5 × 18.0 מ"מ. אנטנה "PCB" מובנית או מחבר "IPEX" (תלוי בגרסה.)

סכמת פינים בבקר

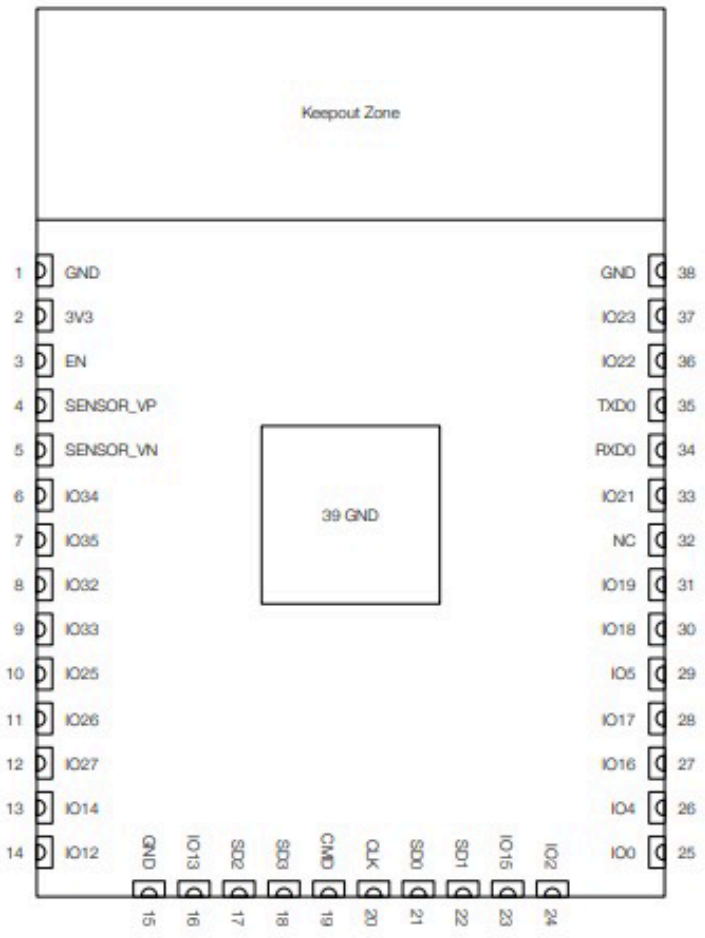


Figure 1: ESP32-WROOM-32D Pin Layout (Top View)

הסבר של כל רכיב

DHT22 module

ה-DHT22 הוא חיישן דיגיטלי נפוץ וחסכוני למדידת טמפרטורה ולחות העובד בתקשורת ONE WIRE. הוא משתמש בחיישן לחות קבל, תרמיסטור למדידת טמפרטורה, ומוציא פלט דיגיטלי דרך ממשק תקשורת חד-גידי (סינגל-ויר). חיישן זה מתאים לשימוש עם פלטפורמות מיקרו-בקר כמו ארדואינו ו-רספברי פיי, ויכול למדוד טמפרטורות בטווח של -40 עד 80 מעלות צלזיוס עם דיוק של ±0.5°C, ולחות בטווח 0-100% עם דיוק של ±2%.

מאפיינים עיקריים

סוג חיישן: מדידת טמפרטורה ולחות דיגיטלית.

טכנולוגיה: משתמש בחיישן לחות קבל ובתרמיסטור.

תקשורת: פלט דיגיטלי דרך קו נתונים יחיד (single-wire).

טווח מתח: 3 עד 5.5 וולט.

טווח טמפרטורה : -40°C עד 80°C .

דיוק טמפרטורה: ±0.5°C .

טווח לחות: 0-100% לחות יחסית (RH).

דיוק לחות: ±2%.

שימוש: מתאים ל-ארדואינו, רספברי פיי, ESP32 ומיקרו-בקרים אחרים.

תפקיד ההדקים

VCC - מתח הזנה לחיישן (בין V3.3 ל- V5.5 בדרך כלל) בלי זה חיישן לא יפעל.

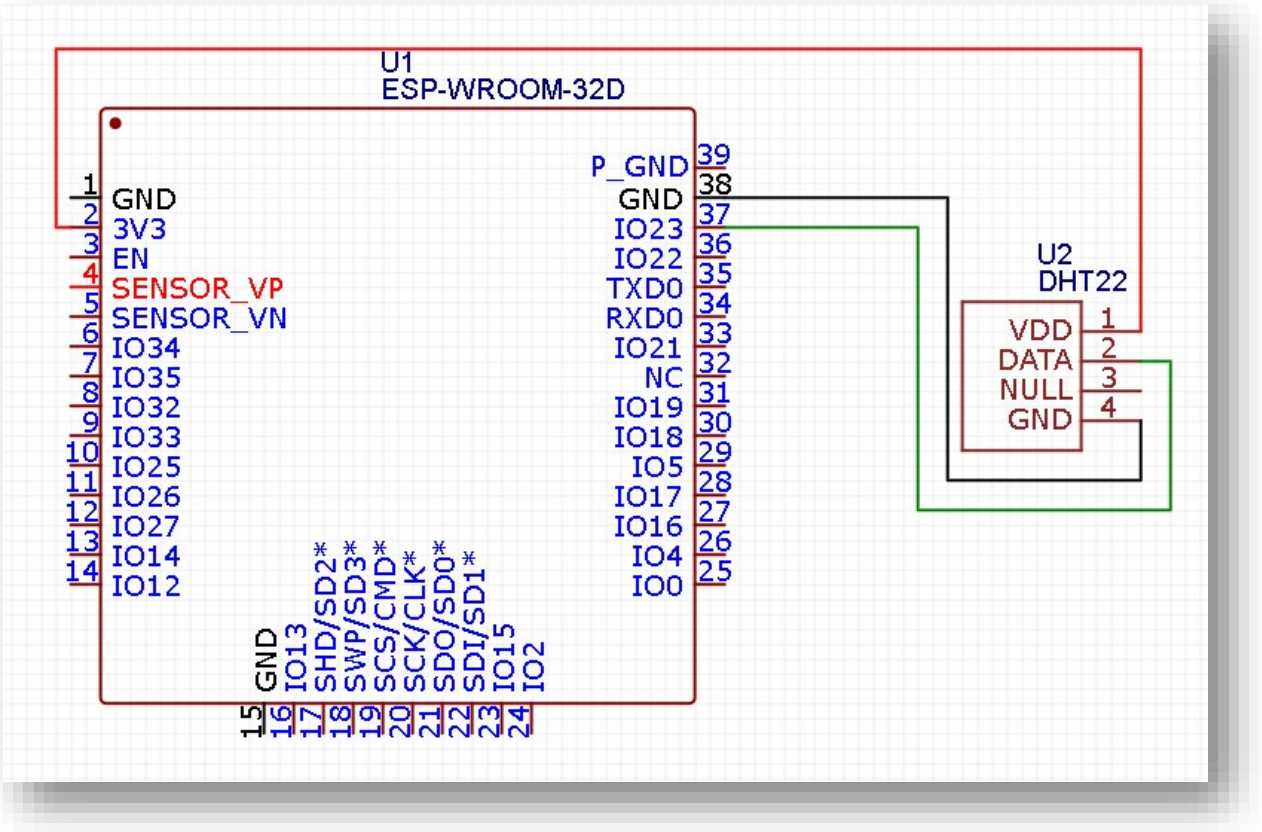
GND - אדמה(GROUND) החיבור להארקה של המעגל.

DATA – קו הנתונים. זה ההדק שדרכו החיישן שולח את ערכי הטמפרטורה והלחות למיקרו-בקר (לדוגמה ESP32 או ARDUINO) הקו הזה הוא דו-כיווני במובן מסוים: הבקר שולח "בקשת קריאה" קצרה ואז החיישן מחזיר תשובה עם המידע.

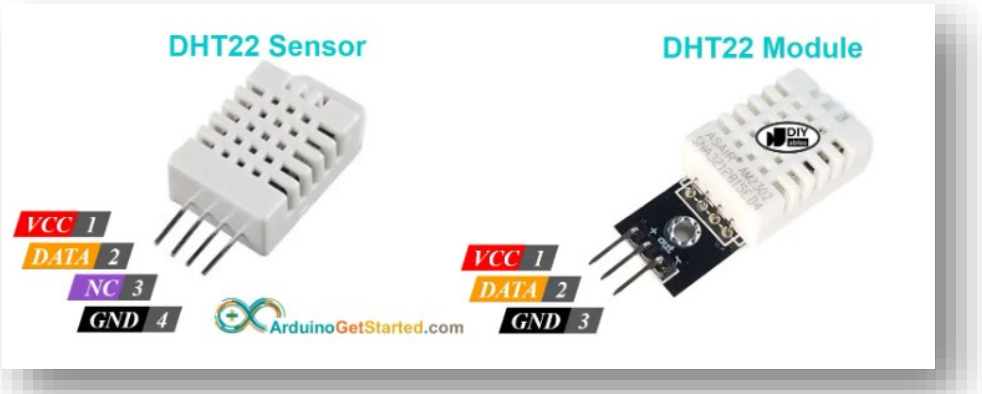
דוגמה לקוד:

```
#include "DHT.h"
#define DHTPIN 4
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);
void setup() {
  Serial.begin(115200);
  Serial.println(F("DHT22 Sensor Test!"));
  dht.begin();
}
void loop() {
  delay(2000);
  float h = dht.readHumidity();
  float t = dht.readTemperature();
  if (isnan(h) || isnan(t)) {
    Serial.println(F("Failed to read from DHT sensor!"));
    return;
  }
  Serial.print(F("Humidity: "));
  Serial.print(h);
  Serial.print(F("%  Temperature: "));
  Serial.print(t);
  Serial.println(F("°C"));
}
```

סכמה חשמלית



תמונה של הרכיב



TFT Touchscreen– מסך טאצ'

המסך הוא מסך ,**TFT (Thin Film Transistor)** שזה סוג של מסך LCD שמאפשר הצגת תמונות צבעוניות ברזולוציה גבוהה. המסך מציג תמונות וטקסט בצורה ברורה, עם צבעים חיים וחדים.

SPI (Serial Peripheral Interface) הוא פרוטוקול תקשורת המאפשר להעביר מידע בצורה סדרתית בין המיקרו-בקר (למשל Arduino או ESP32) למסך.

המסך מאפשר הצגת תמונות, גרפים, אייקונים, טקסט ועוד בצורה ברורה.

מאפיינים עיקריים:

- **גודל מסך**: 2.4 אינץ.
- **רזולוציה**: 240x320 פיקסלים.
- **טכנולוגיית תצוגה**: TFT LCD (מסך צבעוני עם תמיכה בצבעים חיים).
- **התקשורת** : SPI (Serial Peripheral Interface) תקשורת סדרתית בין המיקרו-בקר למסך.
- **הכנסת מידע** : ברוב המקרים, המסך תומך גם בשימוש במסך מגע (Touch Screen),המאפשר אינטראקציה ישירה עם המשתמש.
- **מתח עבודה** : בדרך כלל בין V3.3 ל-V5 .
- **תמיכה בצבעים** : יכול להציג 65,536 צבעים, תלוי במערכת.

תפקיד ההדקים:(PINS)

- **VCC** מחבר את המתח חיובי ישר החשמלי(מ 3.3 עד V5) למערכת.
- **GND** מתח שלילי (אדמה). משמש לחיבור חזרה לאדמה של המערכת.
- **CS (Chip Select)** ניהול הבחירה במכשירSPI אם יש יותר ממסך אחד, אפשר לבחור איזה מכשיר לתקשר איתו.
- **RESET** מחבר להדלקת המכשיר או לאיפוס שלו (הפעלת אתחול מחדש).
- **DC (Data/Command)** שליטה על הבחירה בין שליחת מידע (נתונים) לפקודות למכשיר.
- **SDI (MOSI)** קו המידע (Master Out Slave In) שנשלח אליו מידע מהמיקרו-בקר.
- **SCK** קו שעון (Clock) ששולח את קצב השעון למשדר המידע.

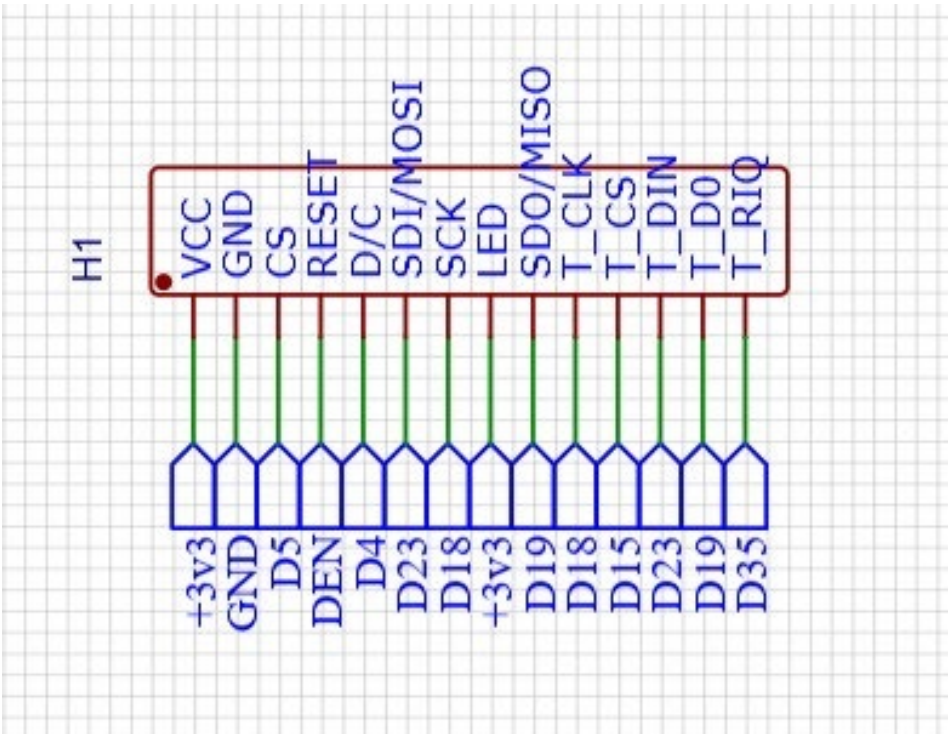
- **LED+** מתח חיובי לנורת ה LED תאורה אחורית של המסך.
- **LED-** מתח שלילי או חיבור לאדמה של תאורת הLED.
- **T_CLK**(לא תמיד נדרש) משדר את קצב השעון למסך המגע במידה ויש.
- **T_CS**(במקרים שיש מסך מגע) קו שליטה נוסף למסך המגע, נועד לאותת שהמסך המגע פעיל.

דוגמה לקוד

```
#include "TFT9341Touch.h"
#define ledBlue 2
tft9341touch lcd(5, 4, 15, 35);
byte flag = 0;
int x, y;
void setup() {
  lcd.begin();
  lcd.setTouch(3780, 372, 489, 3811);
  lcd.print(20, 20, "Test SWITCH", 3, YELLOW, BLACK);
  lcd.drawCircle(150, 120, 40, WHITE);
  lcd.fillRoundRect(110, 180, 80, 50, 5, CYAN);
  lcd.print(130, 190, "ON", 3, BLACK, CYAN);
  pinMode(ledBlue, OUTPUT);
}
void loop() {
  if (lcd.touched()) {
```

```
    delay(30);
    lcd.readTouch();
    x = lcd.xTouch;
    y = lcd.yTouch;
    if ((x > 110) && (x < (110 + 80)) && (y > 180) && (y < (180 + 50))) {
      flag = ~flag;
    }
    if (flag) {
      lcd.fillCircle(150, 120, 39, BLUE);
      digitalWrite(ledBlue, HIGH);
    } else {
      lcd.fillCircle(150, 120, 39, BLACK);
      digitalWrite(ledBlue, LOW);
    }
    while (lcd.touched())
      delay(10);
  }
}
```

סכמה חשמלית



תמונה של הרכיב



WATER LEVEL SENSOR -חיישן גובה מים

מודד לחות מים הוא רכיב אשר מודד את גובה המים שיש על הרכיב באמצעות קווי נחושת שמחוברים לטרנזיסטור ששולח מידע אנלוגי למעבד (מאסטר), ערכי פלס המים נעים בין 0(מינימום) ל־520(מקסימום). הרכיב מראה על המסך את הנתונים שהמודד מדד בתוך מד הגשם הקשור לתחנה המטאורולוגית.

מאפיינים:

- משתמש בכוח של +V5.
- טרנזיסטור NPN 8050S .
- נגד מובנה של 100 אוהם.
- משתמש ב־ ADC(ממיר אנלוגי לדיגיטלי) על מנת לשלוח את המדידות.

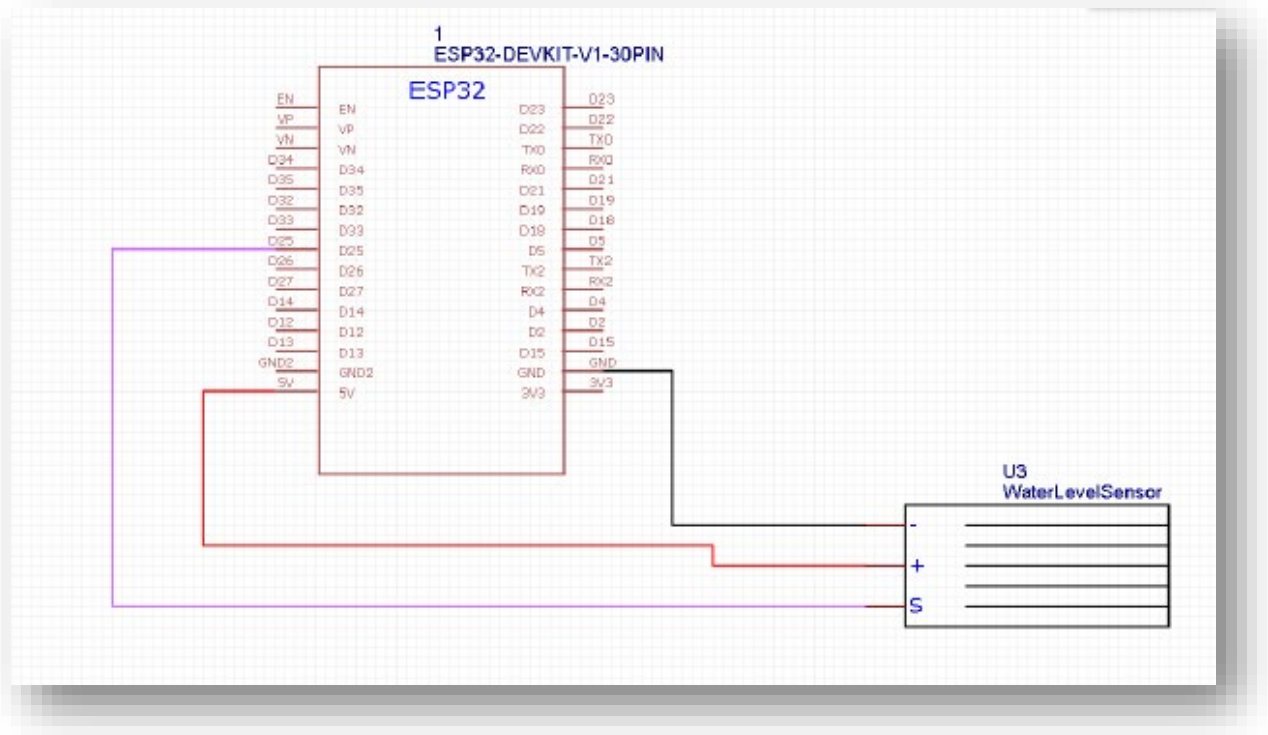
תפקיד ההדקים:

- פלוס: מחובר ל־(+5V) VCC.
- מינוס: מחובר לאדמה(GND) .
- S: הדק סיגנל שמעביר ערך אנלוגי לדיגיטלי. מחובר לרגל דיגיטלית במעבד.

דוגמה לקוד

```
int waterSensorPin = 26;
int waterLevelValue = 0;
void setup() {
  Serial.begin(9600);
}
void loop() {
  waterLevelValue = analogRead(waterSensorPin);
  Serial.print("Water level: ");
  Serial.println(waterLevelValue);
  delay(500);
}
```

סכמה חשמלית



תמונה של הרכיב



LDR MODULE(LM393)

תפקיד כללי:

זהו מודול חיישן אור פשוט ונפוץ, שמאפשר למדוד עוצמת תאורה ולהגיב אליה בצורה חכמה. הוא מבוסס על רכיב שנקרא LDR (נגד תלוי אור), שהתנהגותו משתנה לפי כמות האור: ככל שיש יותר אור – ההתנגדות שלו יורדת, וכשחשוך – ההתנגדות עולה. כך אפשר להמיר אור לאות חשמלי.

המודול לא רק "מרגיש" אור, אלא גם יודע להחליט מתי האור מספיק חזק או חלש כדי לבצע פעולה. ה LDR-יוצר מתח משתנה בהתאם לתאורה, והמתח הזה נשלח לשני מסלולים:

- יציאה אנלוגית (AO) שנותנת ערך רציף של עוצמת האור .
- יציאה דיגיטלית (DO) שמופעלת לפי סף שנקבע מראש .

כאן נכנס הפוטנצמטר – בעזרתו אפשר לקבוע את רמת הרגישות, כלומר באיזה שלב המערכת תחליט ש"חשוך" או "מואר". מגבר ההשוואה (Comparator) משווה בין ערך האור הנמדד לבין הסף הזה, ומוציא אות דיגיטלי בהתאם.

בנוסף, יש נורות:LED

- אחת מראה שהמודול מקבל מתח .
- השנייה מראה אם היציאה הדיגיטלית פעילה.

רכיבים עיקריים ותפקידם:

1. LDR (נגודת תלויה באור) –מזהה את עוצמת האור .
2. פוטנצמטר (בורר רגישות) –קובע את סף האור להפעלה/כיבוי.
3. מגבר השוואה – (Comparator) משווה בין עוצמת האור לסף שקבעת.
4. נורות חיווי – (LEDs) אחת למתח, אחת לפעולת היציאה.
5. לוח PCB עם פינים לחיבור.

שימושים:

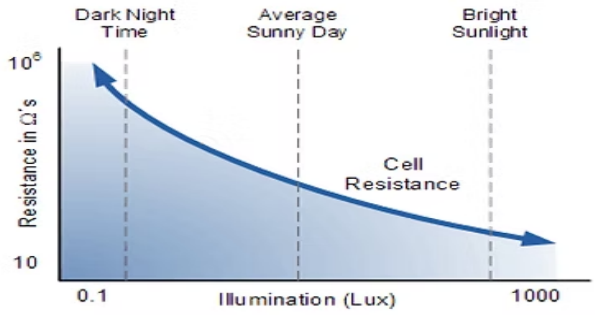
6. הדלקת תאורה אוטומטית בלילה.
7. מדידת עוצמת אור בפרויקטים עם Arduino / Raspberry Pi .
8. מערכות חיסכון באנרגיה או אבטחה.

חיבורי ההדקים (פינים):

1. VCC - מתח הזנה (3.3V–5V) .
2. GND - אדמה (שלילי).
3. AO - יציאה אנלוגית (מתח משתנה לפי עוצמת אור).
4. DO - יציאה דיגיטלית (0/1 לפי הסף שנקבע).

דוגמה לקוד

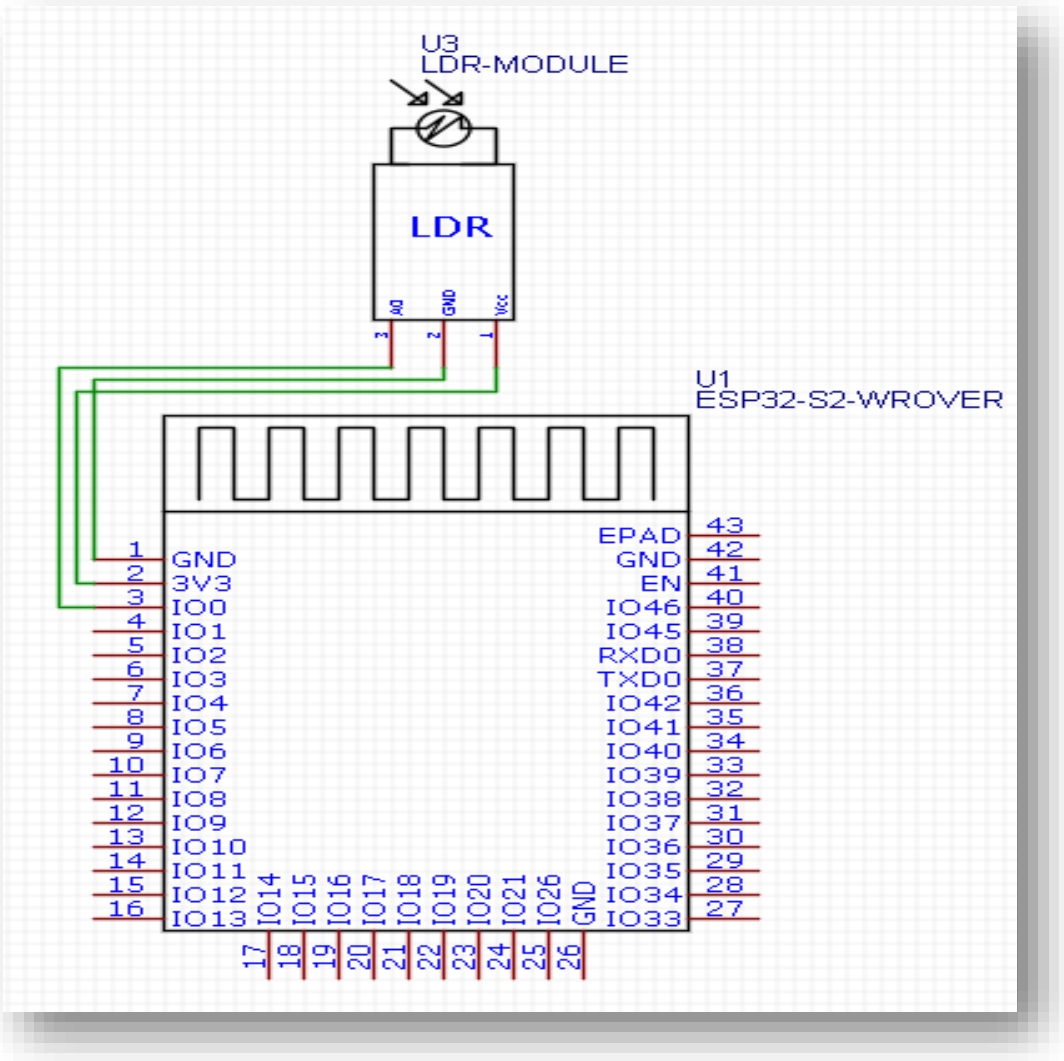
```
int ldrAnalog = 13;
int ledPin = 12;
int threshold = 500;
void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}
void loop() {
  int lightValue = analogRead(ldrAnalog);
  Serial.print("LDR Value: ");
  Serial.println(lightValue);
  if (lightValue < threshold) {
    digitalWrite(ledPin, HIGH);
  } else {
    digitalWrite(ledPin, LOW);
  }
  delay(200);
}
```



ELECTRO-OPTICAL CHARACTERISTICS @ 25° C

Part Number *	Resistance (Ohms) 3						Material Type
	10 lux 2850 K			2 fc 2850 K	DARK		
	Min.	Typ.	Max.	Typ.	Min.	sec.	
VT9ØN1	6.0 k	12 k	18 k	6.0 k	200 k	5	Ø
VT9ØN2	12 k	24 k	36 k	14 k	500 k	5	Ø
VT9ØN3	25 k	50 k	75 k	34 k	1.0 M	5	Ø
VT9ØN4	50 k	100 k	150 k	80 k	2.0 M	5	Ø

סכמה חשמלית



תמונה של הרכיב



MH-RD(Raindrops Module) – חיישן מים

זהו מודול חיישן שמאפשר לזהות נוכחות של מים או גשם על פני משטח. הוא עובד על עיקרון פשוט: כשהלוח המתכתי יבש – ההתנגדות גבוהה; כשהוא נרטב – המים יוצרים מוליכות בין המסלולים, ההתנגדות יורדת, והמערכת מזהה רטיבות.

החיישן מורכב משני חלקים: לוח החישה עצמו (המשטח החשוף למים) ומודול הבקרה. כאשר טיפות מים נוגעות בלוח, הן סוגרות מעגל חשמלי חלקי ומעלות את המוליכות. המודול מתרגם את השינוי הזה לאות חשמלי.

מאפיינים עיקריים

- מתח עבודה: **3.3V–5V** .
- שתי יציאות:
  - **AO** - אות אנלוגי (רמת רטיבות).
  - **DO** - אות דיגיטלי (יבש/רטוב).
- **פוטנציומטר** לכיוון רגישות.
- לוח חישה מצופה ניקל לעמידות.

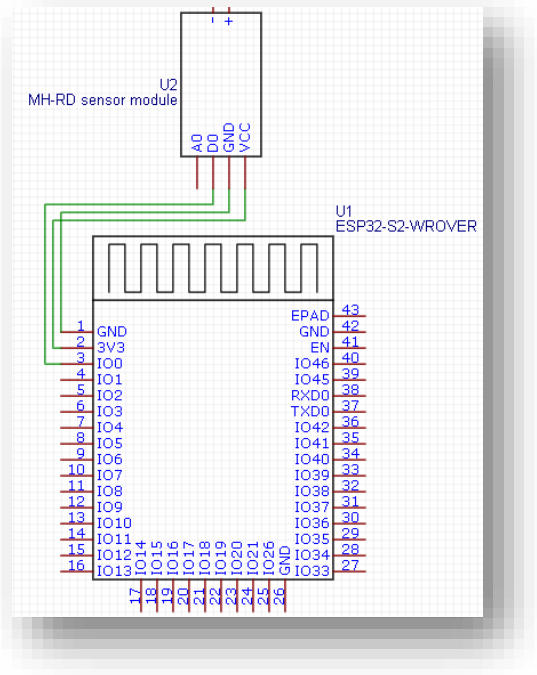
תפקיד ההדקים

- **VCC** - מתח הזנה.
- **GND** – אדמה.
- **AO** - פלט אנלוגי.
- **DO**- פלט דיגיטלי.

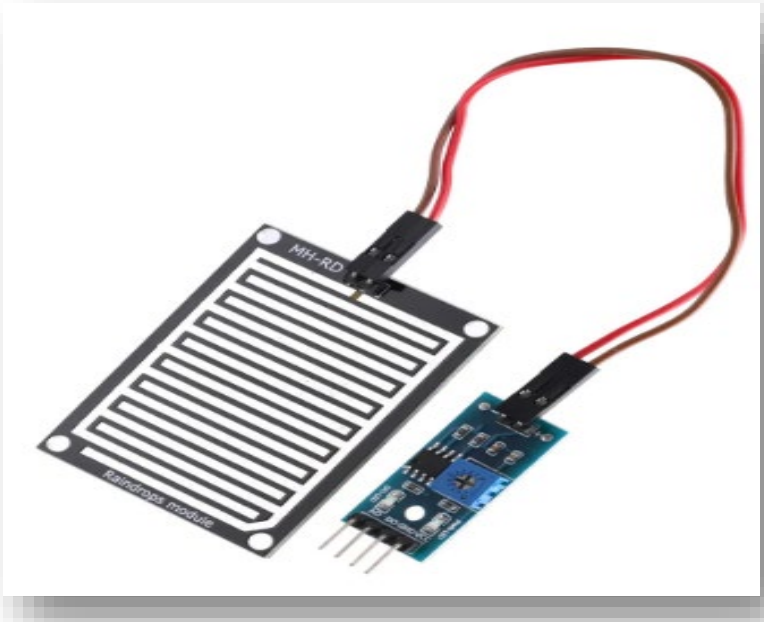
קוד לדוגמה

```
const int sensorPin = 26;
void setup() {
  Serial.begin(9600);
  pinMode(sensorPin, INPUT);
}
void loop() {
  int rainState = digitalRead(sensorPin);
  Serial.println(!rainState);
  delay(50);
}
```

סכמה חשמלית



תמונה של הרכיב



Fan motor (L9110)

L9110 הוא דרייבר זרם קטן שמיועד להפעיל מנועי מאוורר זעירים.(3–5V)
המיקרו־בקר (Arduino/ESP) לא יכול לספק את הזרם שהמאוורר דורש, לכן ה־ L9110 משמש כ"מתווך" שמקבל אות לוגי חלש ומוציא זרם גבוה למאוורר.

מאפיינים עיקריים

- מתאים למנועי מאוורר **3V–5V DC** (מנועים קטנים).
- יכול לספק זרם של כ־ **700–800mA** (תלוי בלוח).
- עובד עם אות לוגי **3.3V או 5V** מתאים ל Arduino / ESP
- תומך ב־ **PWM** לשליטה במהירות המאוורר.
- מאפשר הפעלה בטוחה בלי להעמיס את פיני המיקרו־בקר.
- כולל מגבר זרם פנימי (H-bridge קטן).
- מודול קומפקטי ושקט, מיועד למאווררים ולמנועים זעירים.

פיני המודול ותפקידם

1. הזנת מתח

- **VCC** מספק מתח למנוע ,תלוי במאוורר(3–5V) .
- **GND** אדמה משותפת לבקר ולדרייבר .

2. פיני שליטה (כניסה מהבקר)

במודולי FAN לרוב יש **פין אחד**:

- **IN** הפעלה/כיבוי + שליטה במהירות בעזרת PWM .

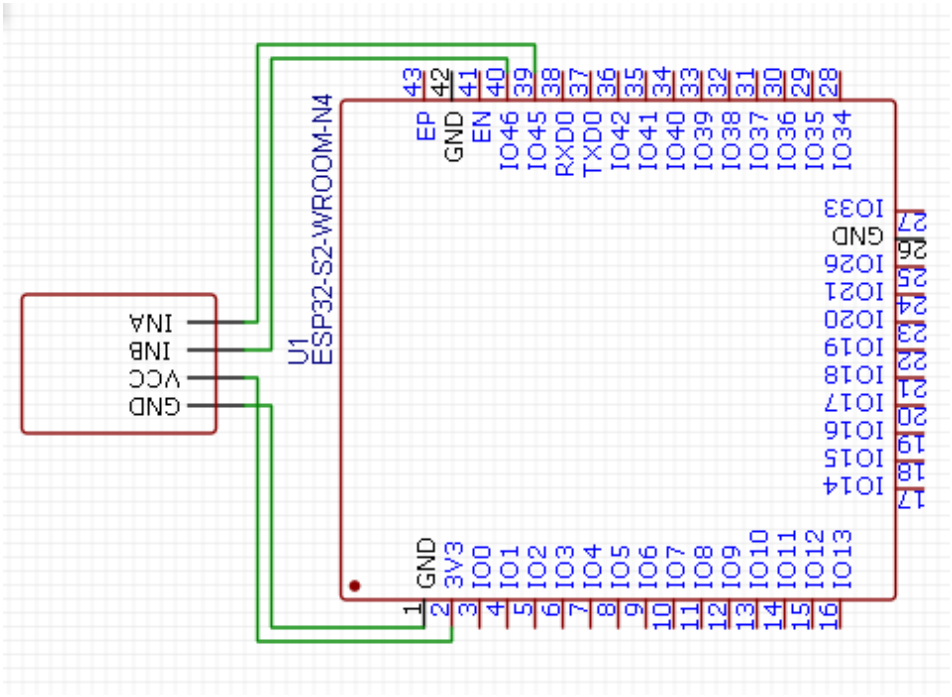
בחלק מהגרסאות יש שני פינים:

- **IN1 / IN2** מאפשרים היפוך כיוון (לרוב לא דרוש למאוורר).

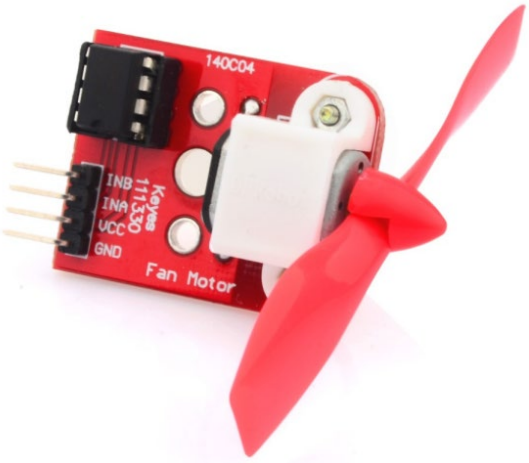
קוד לדוגמה

```
int INA = 13;
int INB = 12;
void setup()
{
   pinMode(INA,OUTPUT);
   pinMode(INB,OUTPUT);
}
void loop()
{
   digitalWrite(INA,LOW);
   digitalWrite(INB,HIGH);
   delay(1000);
}
```

סכמה חשמלית



תמונה של הרכיב



קוד הפרוייקט

```
#include <Adafruit_Sensor.h>
#include <DHT.h>
#include "TFT9341Touch.h"
#include <WiFi.h>
#include <WebServer.h>

/* -------- Pin Definitions -------- */
int INA = 26;              // Motor control pin A
int INB = 25;              // Motor control pin B
#define DHTPIN 13          // DHT22 data pin
#define DHTTYPE DHT22
int waterSensorPin = 14;     // Water presence sensor
int ldrPin = 32;           // Light sensor (ADC1 pin)
int waterLevelSensorPin = 34; // Water level analog sensor

/* -------- Sensor Variables -------- */
int waterLevelValue = 0;
int lightValue = 0;
bool waterValue = 0;
float currentTemp = 0;
float currentHum = 0;

/* -------- WiFi Credentials -------- */
const char* ssid = "Melton-Teachers";
const char* password = "Melton-2018";
```

```
/* -------- Objects -------- */
tft9341touch lcd(5, 4, 15, 35);
WebServer server(80);
DHT dht(DHTPIN, DHTTYPE);

/* -------- Timing Control -------- */
unsigned long lastSensorRead = 0;
const long sensorInterval = 2000;

/* -------- Touch Button Settings -------- */
bool displayMode = false;
int buttonX = 60;
int buttonY = 100;
int buttonW = 200;
int buttonH = 50;

void setup() {
  Serial.begin(9600);
  delay(100);

  // Configure ADC resolution and range
  analogReadResolution(12);
  analogSetAttenuation(ADC_11db);
```

```
  // Initialize display and touch screen
  lcd.begin();
  lcd.setTouch(3780, 372, 489, 3811);
  lcd.setTextSize(1);

  // Configure pin modes
  pinMode(waterLevelSensorPin, INPUT);
  pinMode(waterSensorPin, INPUT);
  pinMode(ldrPin, INPUT);
  pinMode(INA, OUTPUT);
  pinMode(INB, OUTPUT);

  // Initialize system components
  connectToWifi();
  setupServer();
  server.begin();
  dht.begin();

  // Draw the on-screen button
  drawButton();

  Serial.println("System initialized successfully");
}
```

```
/* -------- Draw Touch Button -------- */
void drawButton() {
  lcd.fillRoundRect(buttonX, buttonY, buttonW, buttonH, 5, GREEN);
  lcd.print(buttonX + 20, buttonY + 15, "Change Display", 2, BLACK, GREEN);
}

/* -------- Update Screen Content -------- */
void updateDisplay() {
  lcd.fillRect(0, 0, 320, buttonY - 10, BLACK);

  if (displayMode) {
    lcd.print(20, 20, "Welcome to my", 2, WHITE, BLACK);
    lcd.print(40, 45, "Greenhouse!", 3, GREEN, BLACK);
  } else {
    lcd.print(10, 20, "IP Address:", 2, WHITE, BLACK);
    lcd.print(10, 45, WiFi.localIP().toString(), 2, CYAN, BLACK);
  }
}

void loop() {
  // Handle incoming web requests
  server.handleClient();
```

```
// Check for touch interaction
  if (lcd.touched()) {
    delay(30);
    lcd.readTouch();

    int x = lcd.xTouch;
    int y = lcd.yTouch;

    // Detect button press
    if (x > buttonX && x < (buttonX + buttonW) &&
        y > buttonY && y < (buttonY + buttonH)) {
      displayMode = !displayMode;
      updateDisplay();
    }

    // Wait until touch is released
    while (lcd.touched());
    delay(10);
  }
```

```
// Read sensors at fixed intervals
  unsigned long currentMillis = millis();
  if (currentMillis - lastSensorRead >= sensorInterval) {
    lastSensorRead = currentMillis;

    lightValue = analogRead(ldrPin);
    currentTemp = dht.readTemperature();
    currentHum = dht.readHumidity();
    waterValue = digitalRead(waterSensorPin);
    waterLevelValue = analogRead(waterLevelSensorPin);

    // Automatic motor control based on temperature
    if (!isnan(currentTemp)) {
      if (currentTemp > 25) {
        digitalWrite(INA, LOW);
        digitalWrite(INB, HIGH);
      } else {
        digitalWrite(INA, LOW);
        digitalWrite(INB, LOW);
      }
    }
```

```
 // Serial monitoring for debugging
   Serial.print("Temp: ");
   Serial.print(currentTemp);
   Serial.print(" | Humidity: ");
   Serial.print(currentHum);
   Serial.print(" | Water Detected: ");
   Serial.print(!waterValue);
   Serial.print(" | Light Level: ");
   Serial.print(lightValue);
   Serial.print(" | Water Level: ");
   Serial.println(waterLevelValue);
  }
}


/* -------- HTTP Server Response -------- */
void sendResponse() {
  float hum = currentHum;
  float temp = currentTemp;

  // Validate sensor readings
  if (isnan(hum) || isnan(temp)) {
    server.send(500, "text/plain", "Failed to read from DHT sensor!");
    return;
  }
```

```
  // Build dynamic HTML page with sensor data
  String htmlPage = R"rawliteral(
<!DOCTYPE html><html><head><meta charset='UTF-8'>
<meta http-equiv='refresh' content='3'>
<title>Greenhouse Data</title>
<style>
  body { font-family: Arial, sans-serif; margin: 20px; background-color: #f0f0f0; }
  .container { display: flex; flex-wrap: wrap; gap: 20px; }
  .sensor-section { background-color: white; padding: 20px; border-radius:
10px;
                box-shadow: 0 2px 5px rgba(0,0,0,0.1); flex: 1; min-width: 250px; }
  .top-row { display: flex; gap: 20px; margin-bottom: 20px; }
  .left-column { flex: 1; }
  .center-column { flex: 1; text-align: center; }
  h1 { color: #333; margin-top: 0; font-size: 1.5em; }
  h2 { color: #666; font-size: 1.1em; }
  img { margin: 10px 0; }
</style>
</head><body>


<div class='top-row'>
  <div class='sensor-section left-column'>
    <h1>Raindrops Module:</h1>
    <img src=https://encrypted-
tbn0.gstatic.com/images?q=tbn:ANd9GcQpNspr1antNGAkaaCb07VY_C3RYMG
WF50YAQ&s' width='150'>
```

```
  <h2>Presence of Water: )rawliteral" + String(!waterValue) +
R"rawliteral(</h2>

 </div>


 <div class='sensor-section center-column'>

  <h1>Light Sensor:</h1>

  <img src='https://see-sys.co.il/wp-
content/uploads/2025/03/ldr_sensor_module_1_.jpg' width='150'>

  <h2>Light Level: )rawliteral" + String(lightValue < 2000 ? "Good" : "Not
Good") + R"rawliteral(</h2>

 </div>

</div>


<div class='top-row'>

 <div class='sensor-section left-column'>

  <h1>DHT22:</h1>

  <img src='https://o.lnwfile.com/_/o/_raw/zp/f9/qc.png' width='150'>

  <h2>Temperature: )rawliteral" + String(temp, 1) + R"rawliteral( °C</h2>

  <h2>Humidity: )rawliteral" + String(hum, 1) + R"rawliteral( %</h2>

  <h2>Motor Status: )rawliteral" + String(temp > 25 ? "ON" : "OFF") +
R"rawliteral(</h2>

 </div>


 <div class='sensor-section center-column'>

  <h1>Water Level Sensor:</h1>

  <img src='https://encrypted-
tbn0.gstatic.com/images?q=tbn:ANd9GcQqiwcOUQKCWqRZcUqFUMTOnlqUz9
5j7Vn3eQ&s' width='150'>
```

```
  <h2>Status: )rawliteral" + String(waterLevelValue > 2500 ? "Good" : "Bad") +
R"rawliteral(</h2>
 </div>
</div>

</body></html>
)rawliteral";

 // Send HTML page to client
 server.send(200, "text/html", htmlPage);
}

/* -------- Web Server Setup -------- */
void setupServer() {
 server.on("/", HTTP_GET, []() {
  sendResponse();
 });
}

/* -------- WiFi Connection Function -------- */
void connectToWifi() {

 // Display WiFi connection status on screen
 lcd.print(5, 5, "Connecting to WiFi", 2, WHITE, BLACK);
 lcd.print(5, 25, ".", 2, WHITE, BLACK);
```

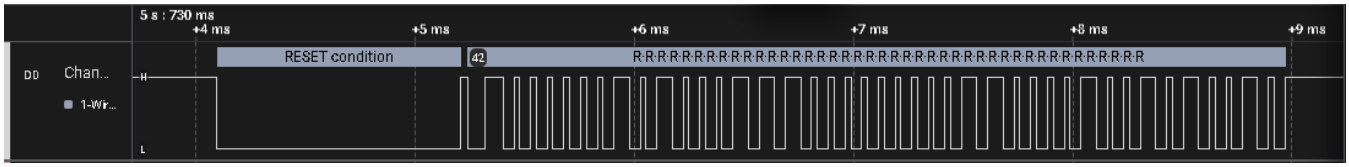
```
  // Start WiFi connection
  WiFi.begin(ssid, password);
  // Wait until WiFi is connected
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    lcd.print(".");
  }

  // Clear screen after successful connection
  lcd.fillScreen(BLACK);
  lcd.print(10, 20, "Connected to WiFi", 2, GREEN, BLACK);
  delay(1000);

  // Display initial screen content (IP address by default)
  updateDisplay();
}
```

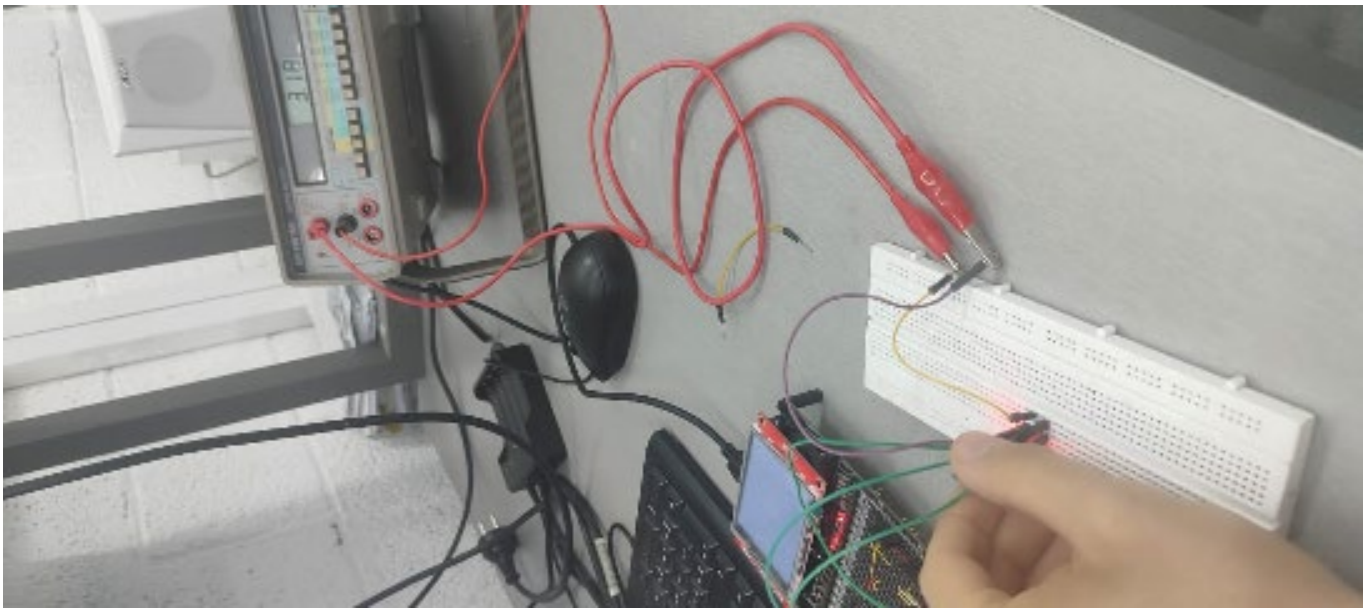
מדידות של הרכיבים

מדידת פרוטוקול 1-Wire בחיישן DHT22

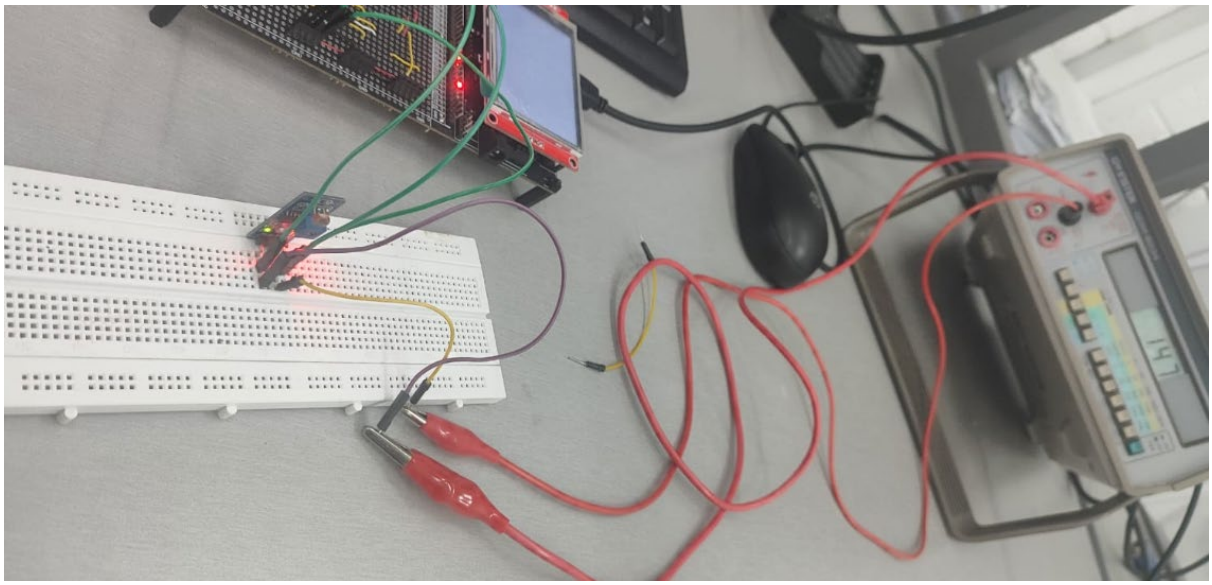


מדידת מתח על החיישן LDR כאשר כשהאור חזק המתח הוא 1.50~ וכשהאור חלש 3.20~

לפני האור החזק:

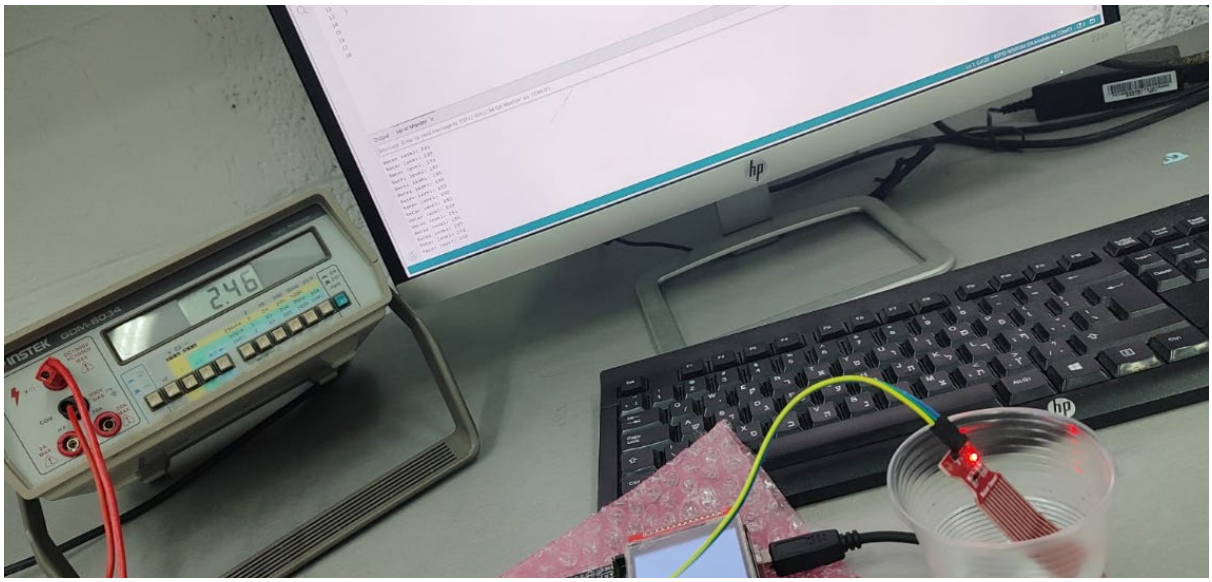


אחרי האור החזק:

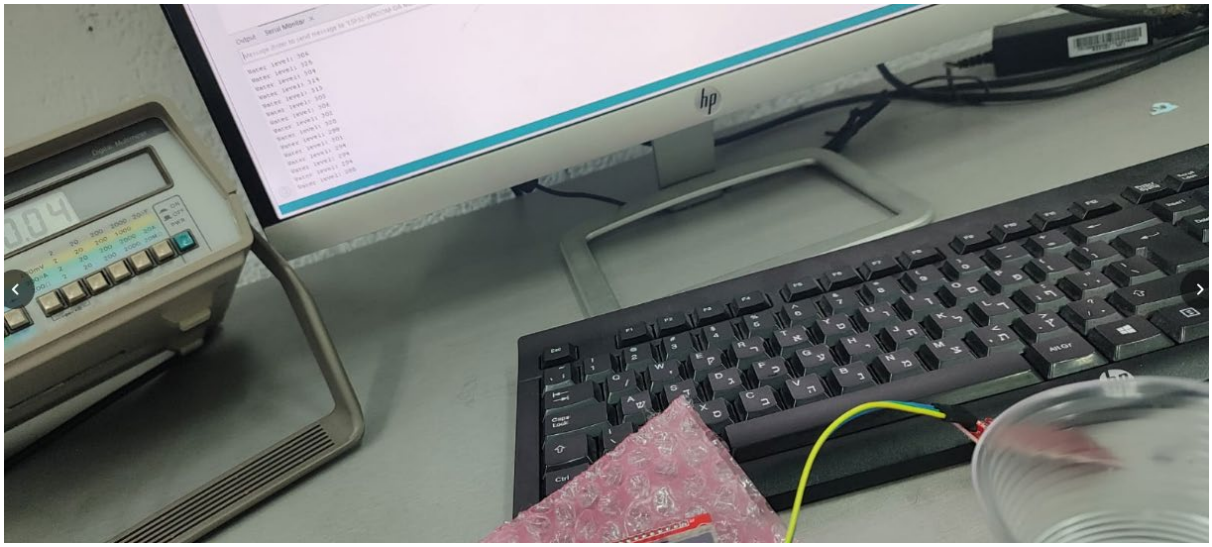


מדידת מתח על החיישן WATER LEVEL כאשר כשהוא נמצא במים המתח הוא 2.50~ וכשהוא נמצא מחוץ למים 0~

התוצאה במים:

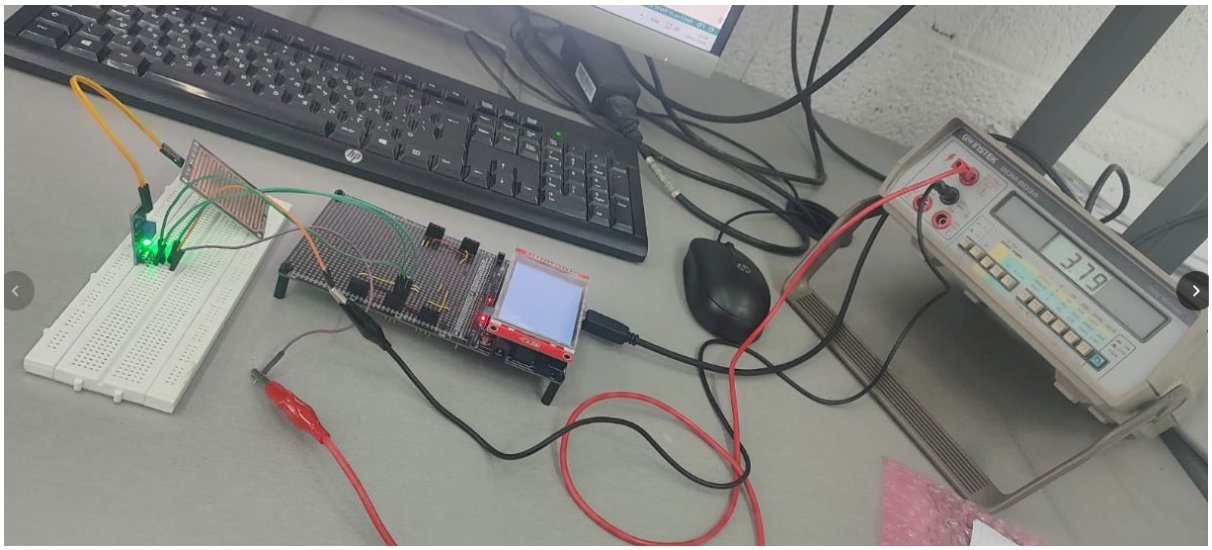


התוצאה מחוץ למים:

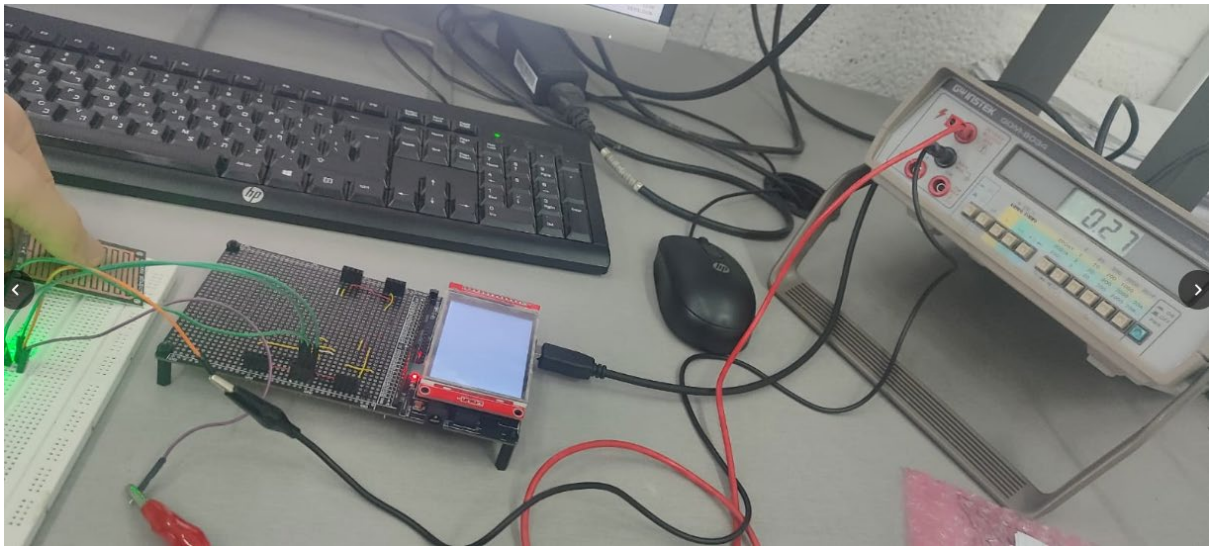


<u>מדידת מתח על החיישן WATER PRESENCE SENSOR כאשר כשנוגע בו מים הוא 2.50~ וכשלא נוגע 0~</u>

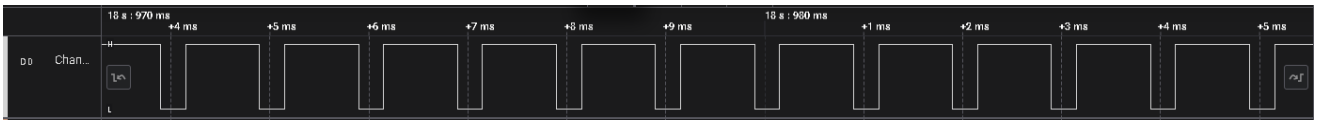
<u>התוצאה בלי מים:</u>



<u>התוצאה עם מים:</u>



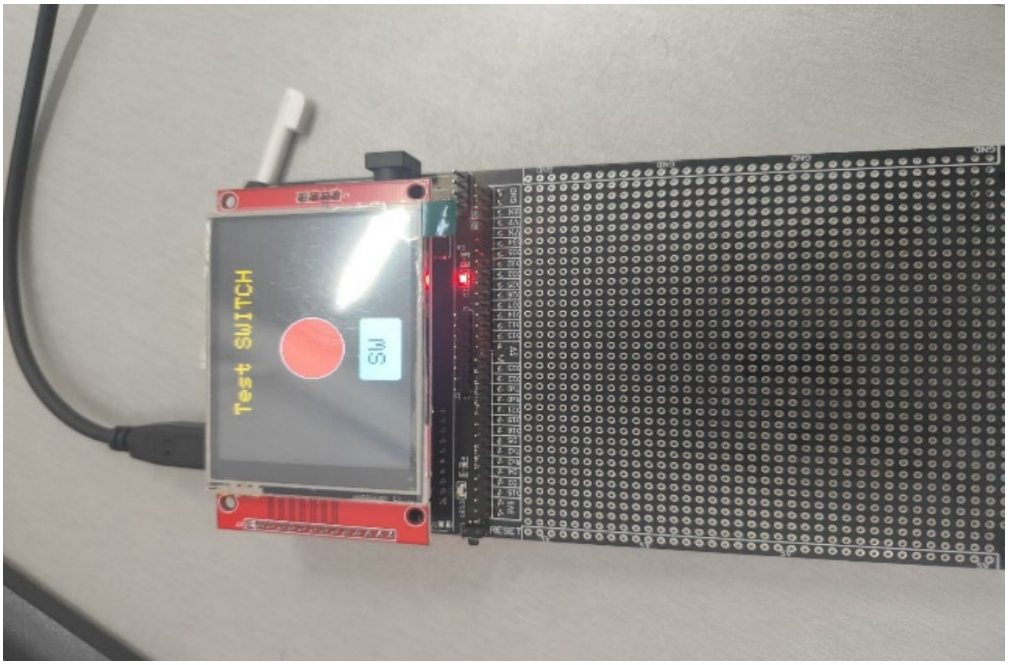
מדידת PWM בDC MOTOR FAN:

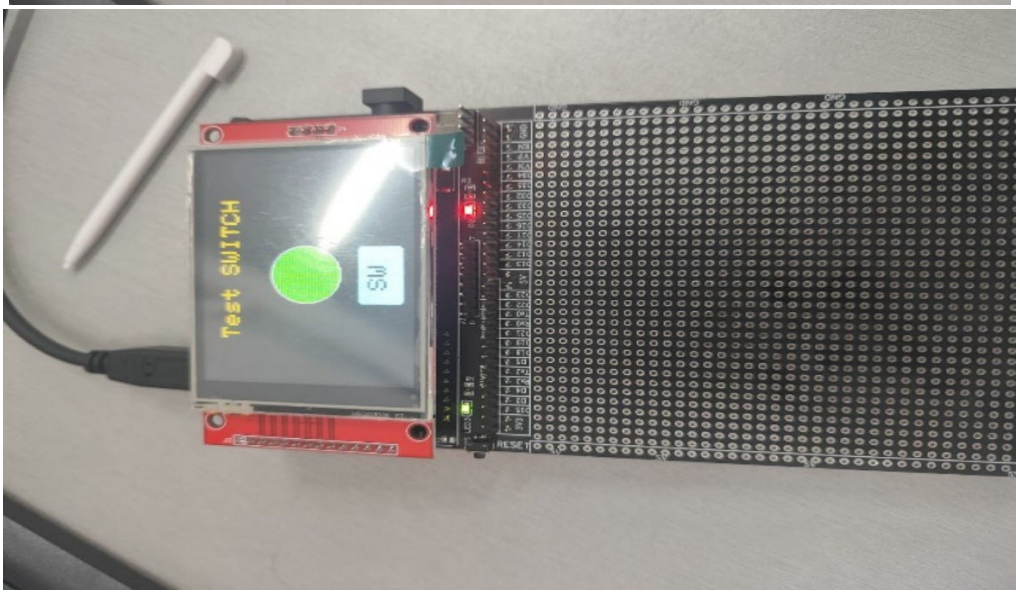


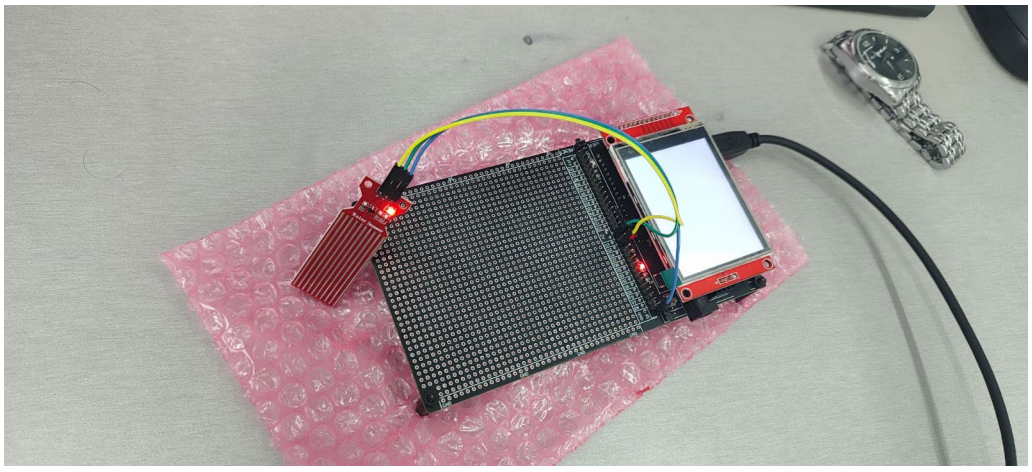
תיעוד

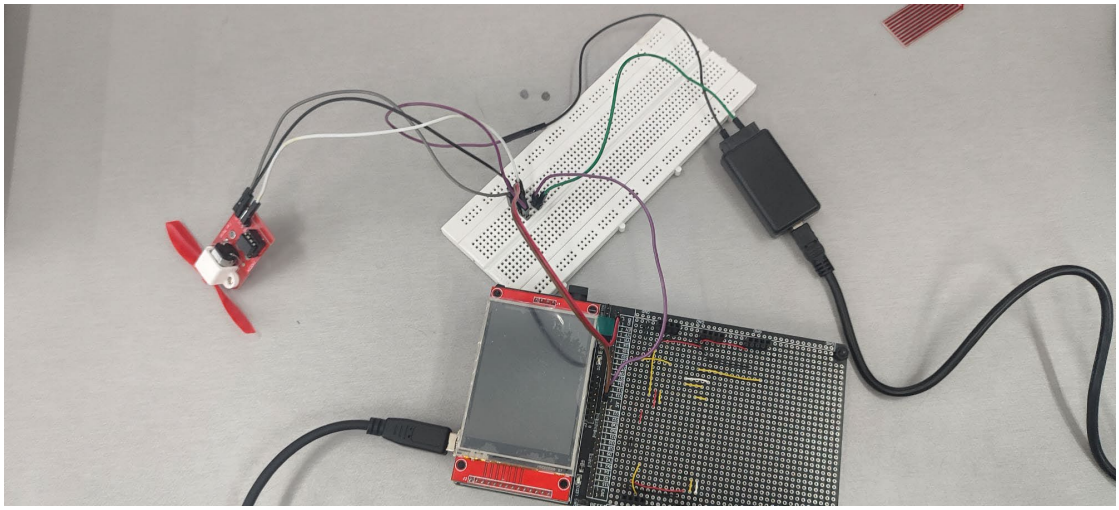


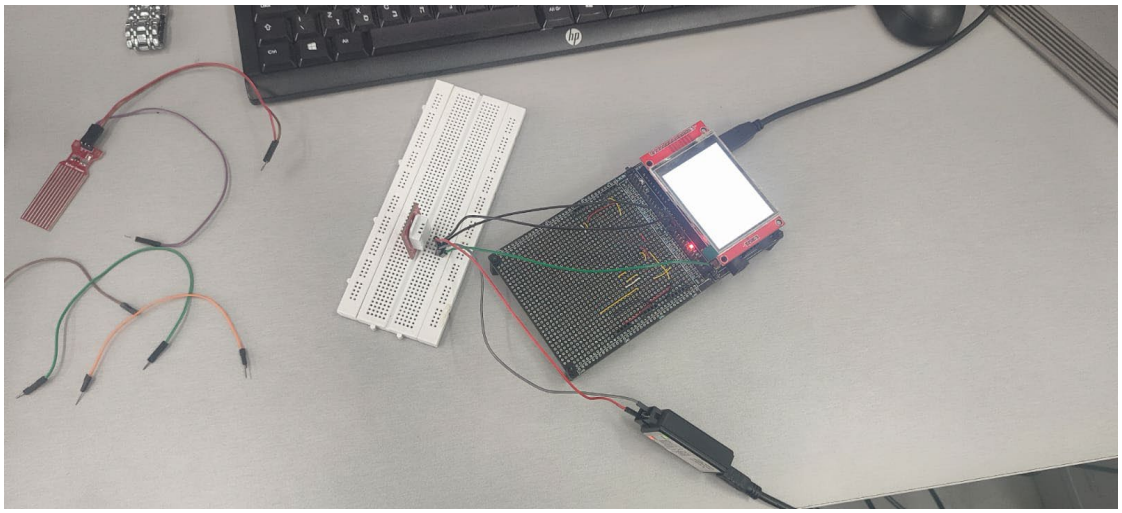


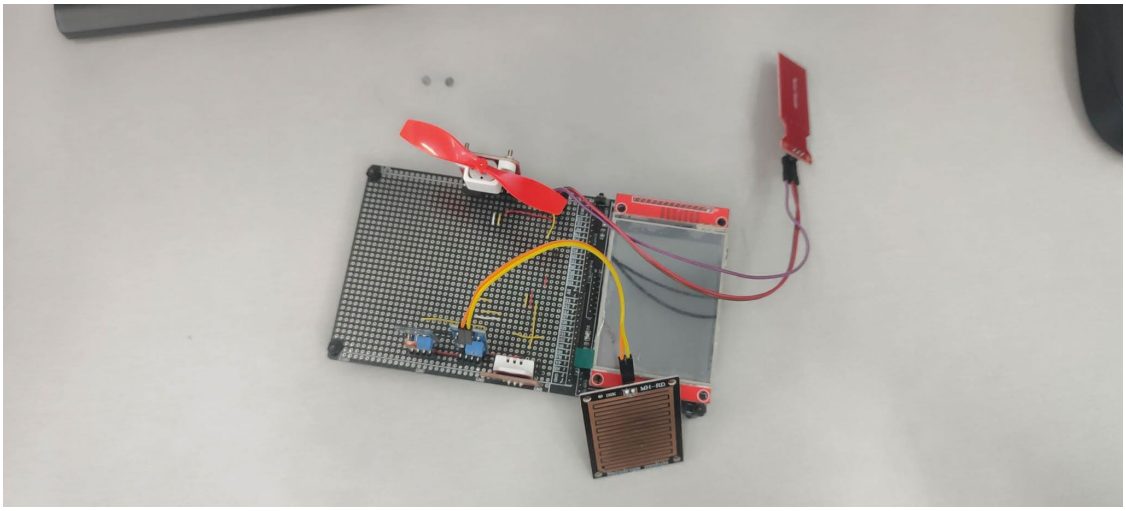

Test SWITCH
SW

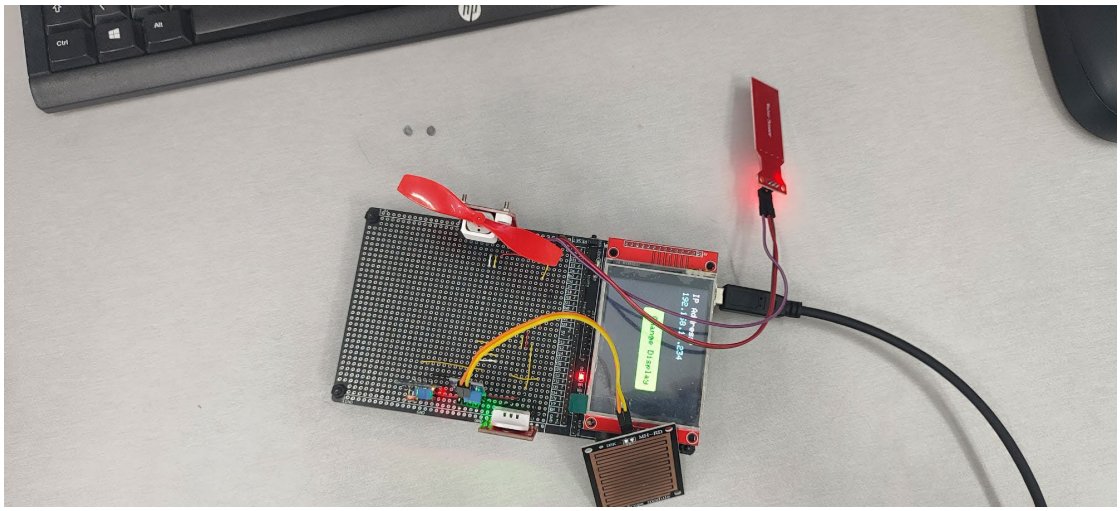

Test SWITCH
SW











סיכום ומסכנות

במהלך העבודה על פרויקט הגמר באלקטרוניקה עברתי תהליך לימודי משמעותי, הן מבחינה מקצועית והן מבחינה אישית. הפרויקט דרש ממני לשלב ידע תיאורטי שנלמד לאורך השנים עם יישום מעשי, תכנון, בדיקות ופתרון בעיות בזמן אמת.

בשלב התכנון נתקלתי בקשיים בהבנת אופן פעולת המערכת ובבחירת הרכיבים המתאימים. נדרשתי לקרוא חומר נוסף, להתייעץ עם המורה ולבצע ניסויים חוזרים עד שהצלחתי להגיע לפתרון תקין. שלב זה חיזק אצלי את ההבנה שאין פתרון מיידי לכל בעיה, ושלעיתים נדרשת סבלנות והתמדה כדי להתקדם.

בשלב הבנייה והבדיקות למדתי לעבוד בצורה מסודרת ומדויקת יותר. טעויות בחיבורים או בתכנות הובילו לתקלות, אך כל תקלה הייתה הזדמנות ללמוד, להבין מה השתבש ולשפר את העבודה. עם הזמן פיתחתי יכולת טובה יותר לאיתור תקלות (debugging)ולעבודה עצמאית.

מעבר לידע הטכני, הפרויקט תרם לי גם בפן האישי. למדתי לנהל זמן, להציב יעדים, לעבוד בצורה עצמאית ולקחת אחריות על תהליך העבודה כולו. תחושת הסיפוק שהרגשתי כאשר המערכת פעלה כמצופה הייתה גדולה והוכיחה לי שהמאמץ השתלם.

לסיכום, פרויקט הגמר באלקטרוניקה היה חוויה מאתגרת ומלמדת, שתרמה לי ידע, כלים וביטחון עצמי להמשך הדרך, הן בלימודים עתידיים והן בעולם הטכנולוגי בכלל.

קשיים

הבנת סיפריית הWIFI

מימוש מחרוזת גולמית

בילבול פינים בwire up

בעיה בADC

ביבליוגרפיה

אלקטרוניקה ומחשבים - יצחק, ברקת ושוהם כהן

ESP32 - Wikipedia

https://he.wikipedia.org/wiki/ADC

https://en.wikipedia.org/wiki/Pulse-width_modulation

/https://see-sys.co.il